A Mobile Robot Application and Transition Systems

Instructor: Dr. Tanmoy Kundu
Dept. of CSE, IIIT-Delhi

September 1, 2026

Wide Range of Applications of Autonomous Robots

- Autonomous driving
- Indoor, urban, under-water navigation
- Exploring space and other planets
- Disaster response
- Manifold applications of drones
- Domestic chores
- Multi-robot, swarms
- ... and many more.

Safety properties

Safety Properties

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations
  - In the context of systems

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations
  - In the context of systems
    - Violation of Mutual Exclusion for more than one more processes with shared resources
    - In Dining Philosopher problem all philosophers waiting to pick up the second chopstick

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations
  - In the context of systems
    - Violation of Mutual Exclusion for more than one more processes with shared resources
    - In Dining Philosopher problem all philosophers waiting to pick up the second chopstick
  - In the context of robot motion planning

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations
  - In the context of systems
    - Violation of Mutual Exclusion for more than one more processes with shared resources
    - In Dining Philosopher problem all philosophers waiting to pick up the second chopstick
  - In the context of robot motion planning
    - Colliding to obstacles or other robots

Safety Properties

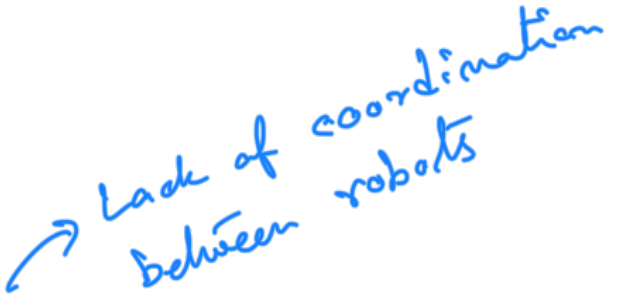
- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations
  - In the context of systems
    - Violation of Mutual Exclusion for more than one more processes with shared resources
    - In Dining Philosopher problem all philosophers waiting to pick up the second chopstick
  - In the context of robot motion planning
    - Colliding to obstacles or other robots
    - Violating traffic lights

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations
  - In the context of systems
    - Violation of Mutual Exclusion for more than one more processes with shared resources
    - In Dining Philosopher problem all philosophers waiting to pick up the second chopstick
  - In the context of robot motion planning
    - Colliding to obstacles or other robots
    - Violating traffic lights
    - Running out of battery charge

Safety Properties

- Safety property $\Rightarrow$ something bad should never happen
- Examples of "Bad" situations
  - In the context of systems
    - Violation of Mutual Exclusion for more than one more processes with shared resources
    - In Dining Philosopher problem all philosophers waiting to pick up the second chopstick
  - In the context of robot motion planning
    - Colliding to obstacles or other robots
    - Violating traffic lights
    - Running out of battery charge
    - Inconsistent action selection by cooperative robots



Example of Robot Planning

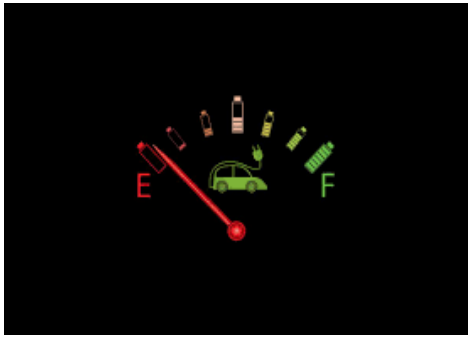
Energy-safety for Battery recharging

Energy-Safe and Unsafe states – Battery recharging



Energy-Safe and Unsafe states – Battery recharging





Energy-Safe and Unsafe states – Battery recharging







Energy-Safe and Unsafe states – Battery recharging









Energy-Safety for Battery Recharging

- Energy-Unsafe state: A robot operational for an extended duration may run entirely out of charge and become non-functional.
- Efficient approaches are required to circumvent Energy-unsafe states

Energy-Safety for Battery Recharging

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state
  - What if the energy-safety is trivially satisfied ?

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state
  - What if the energy-safety is trivially satisfied ?
- **Liveness** with energy-safety

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state
  - What if the energy-safety is trivially satisfied ?
- **Liveness** with energy-safety
  - Robot performs its task along with energy-safety

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state
  - What if the energy-safety is trivially satisfied ?
- **Liveness** with energy-safety
  - Robot performs its task along with energy-safety
- **Performance optimization** with energy-safety

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state
  - What if the energy-safety is trivially satisfied ?
- **Liveness** with energy-safety
  - Robot performs its task along with energy-safety
- **Performance optimization** with energy-safety
  - Minimize path cost

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state
  - What if the energy-safety is trivially satisfied ?
- **Liveness** with energy-safety
  - Robot performs its task along with energy-safety
- **Performance optimization** with energy-safety
  - Minimize path cost
  - Minimize wait time

Energy-Safety for Battery Recharging

- **Energy-safety:** Ensures that the robots never runs into an Energy-unsafe state
  - What if the energy-safety is trivially satisfied ?
- **Liveness** with energy-safety
  - Robot performs its task along with energy-safety
- **Performance optimization** with energy-safety
  - Minimize path cost
  - Minimize wait time
  - Minimize number of charging stations in a workspace

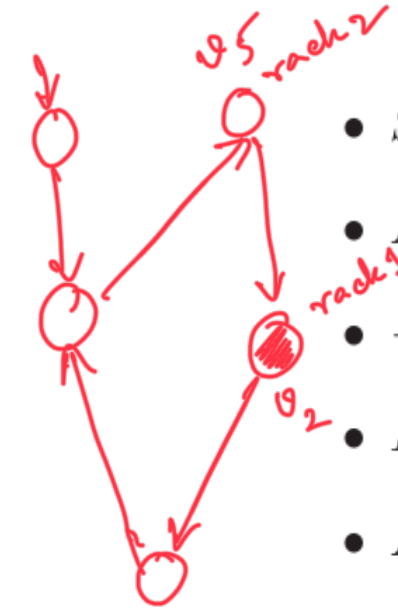
An introduction to Transition Systems

An introduction to Transition Systems

Principles of Model checking by Kaloen and Baier MIT Press.

Definition 2.1. Transition System (TS)

A *transition system* TS is a tuple $(S, Act, \rightarrow, I, AP, L)$ where



- S is a set of states,
- Act is a set of actions,
- $\longrightarrow \subseteq S \times Act \times S$ is a transition relation,
- $I \subseteq S$ is a set of initial states,
- AP is a set of atomic propositions, and
- $L : S \rightarrow 2^{AP}$ is a labeling function.

$s_i \xrightarrow{a_k} s_j$

For a warehouse example, $AP = \{rack_1, rack_2, \ldots\}$

TS is called *finite* if S, Act, and AP are finite.

$L(v_2) = \{rack_1\}$

$L(v_5) = \{rack_2\}$

Remind

Given a workspace W, robot dynamics Γ, create a graph.

- Try to understand the difference with a graph and TS.

Mobile robots as Transition Systems

A mobile robot in a workspace $\longrightarrow$ TS

- useful in planning (motion, task), multi-robot coordination

Given a workspace, robot dynamics and semantic of the states (meaning/AP)

$$TS_R = (S, Act, \longrightarrow, \ldots)$$

Extension to multiple mobile robots

For robots r_1, r_2

$TS_1 = (S_1, Act, \rightarrow, \ldots)$

$TS_2 = (S_2, Act \rightarrow, \ldots)$

$TS_{joint} = TS_1 \otimes TS_2$ (??)

↳ A joint state (s_i, s_j) where $s_i \in S_1$ and $s_j \in S_2$

No. of joint states

$O(|S_1| \cdot |S_2|)$

For an no. of robots

$O(|S|^n)$

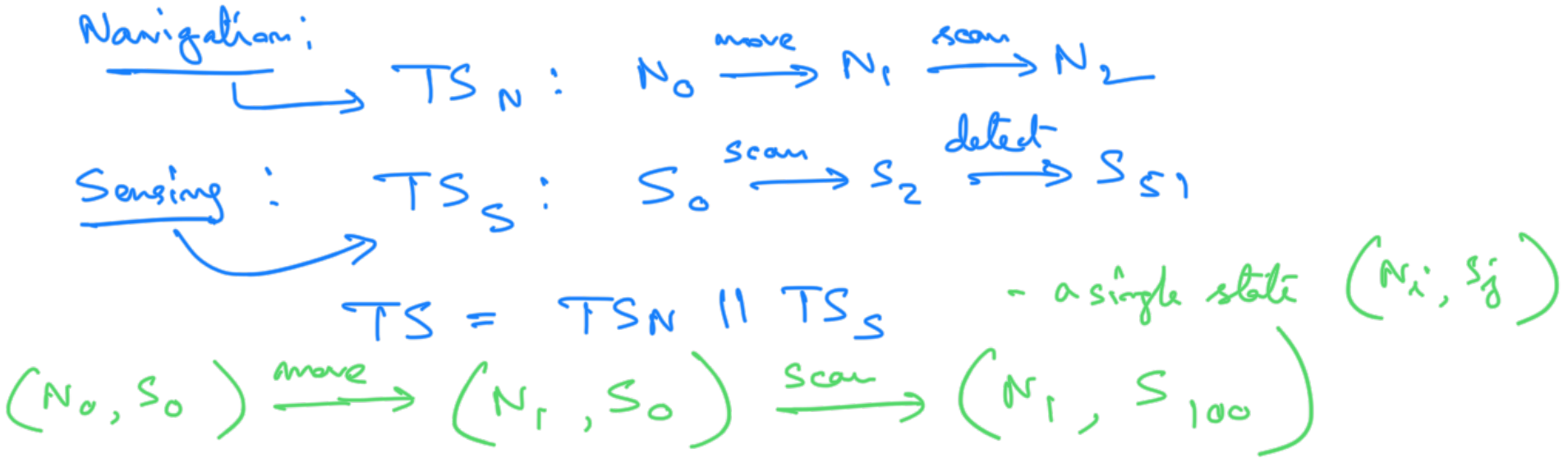
This is where parallelism and communication come into picture.

Extension to multiple mobile robots

Parallelism in robot processes

First assignment: Implementation of a robotic application in ROS 2
- Safety / reachability / deadlock freedom / prob. guarantees.

1. A real robot system is naturally a *parallel system*
   - Navigation, sensing, communication, battery monitoring, safety control etc. can all execute concurrently.

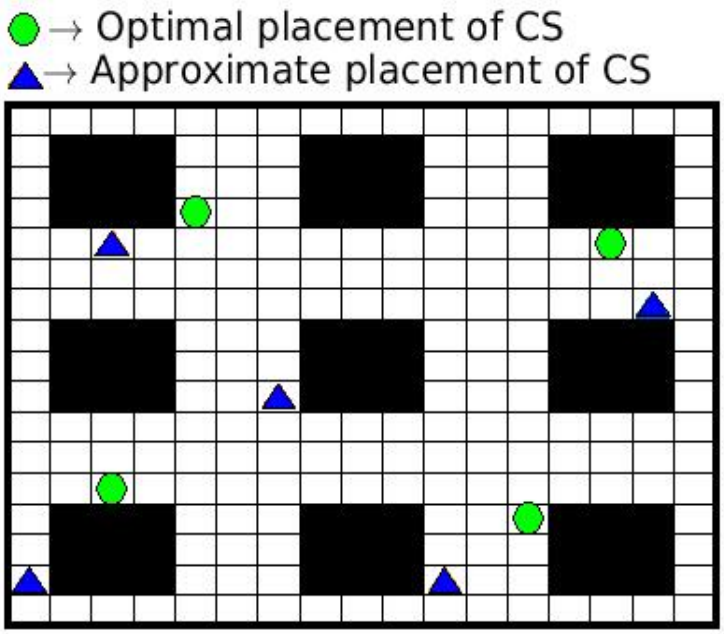


Parallelism in robot processes

1. **Charging station placement**
   - **Counter-example guided approach** (model checking)

Charging Station Placement Problem (CSPP)

- Optimal placement of charging stations in a workspace
- When battery charge of a robot reaches a certain threshold the robot should reach a charging station (with a pre-decided battery threshold)
- Find charging station locations such that at least one charging station is reachable with battery threshold (d) amount of charge remaining with the robot



Charging Station Placement vs Facility location problem

[1]H.W. Hamacher and Z. Drezner, "Facility Location: Applications and Theory", Springer, 2002

Charging Station Placement vs Facility location problem

- CSPP resembles well known *Facility Location Problem (FLP)* [1]

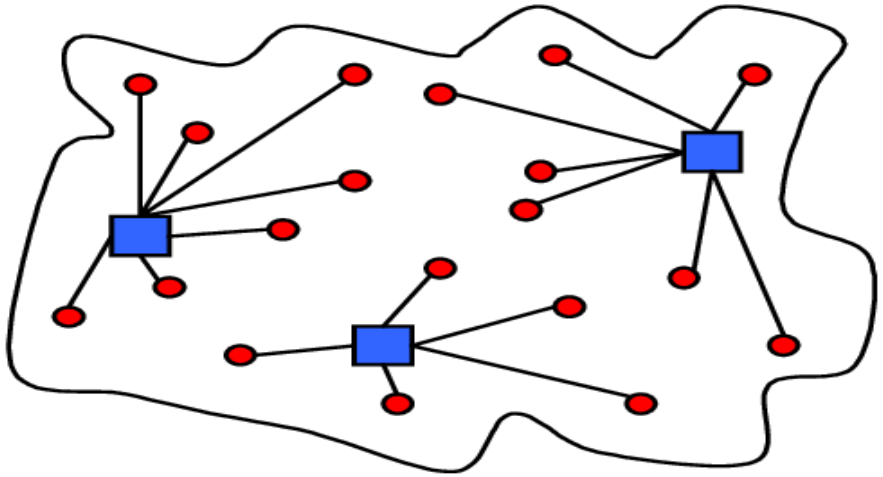


Figure: FLP (Image source: Google)

[1] H.W. Hamacher and Z. Drezner, "Facility Location: Applications and Theory", Springer, 2002

Charging Station Placement vs Facility location problem

- CSPP resembles well known *Facility Location Problem (FLP)* [1]

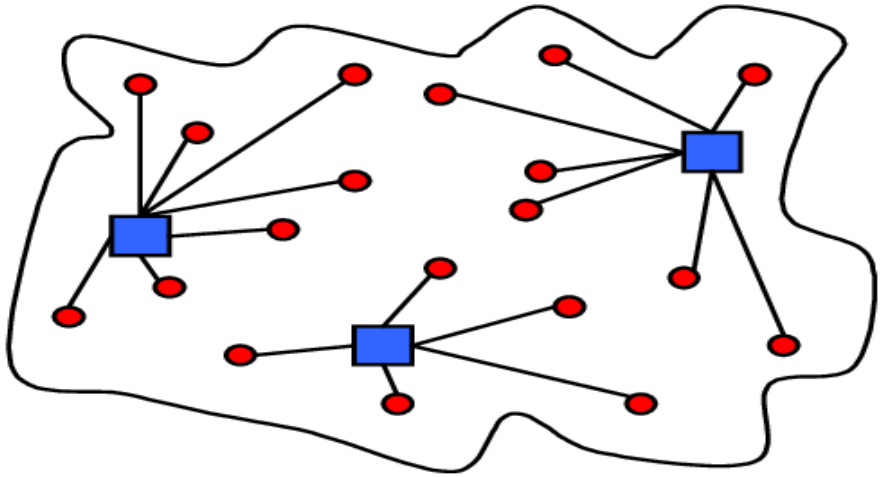


Figure: FLP (Image source: Google)

- FLP considers Euclidean distance only. Does not consider complex dynamics of agents, complex workspaces and trajectories of the agents.

[1] H.W. Hamacher and Z. Drezner, "Facility Location: Applications and Theory", Springer, 2002

Charging Station Placement vs Facility Location Problem

Charging Station Placement vs Facility Location Problem

- Charging Station Placement problem bridges the gap

Charging Station Placement vs Facility Location Problem

- Charging Station Placement problem bridges the gap
  - Complex dynamics of robots
  - Complex design of workspaces with obstacles
  - Finds trajectories of the robots from any location in the workspace towards the nearest charging station location

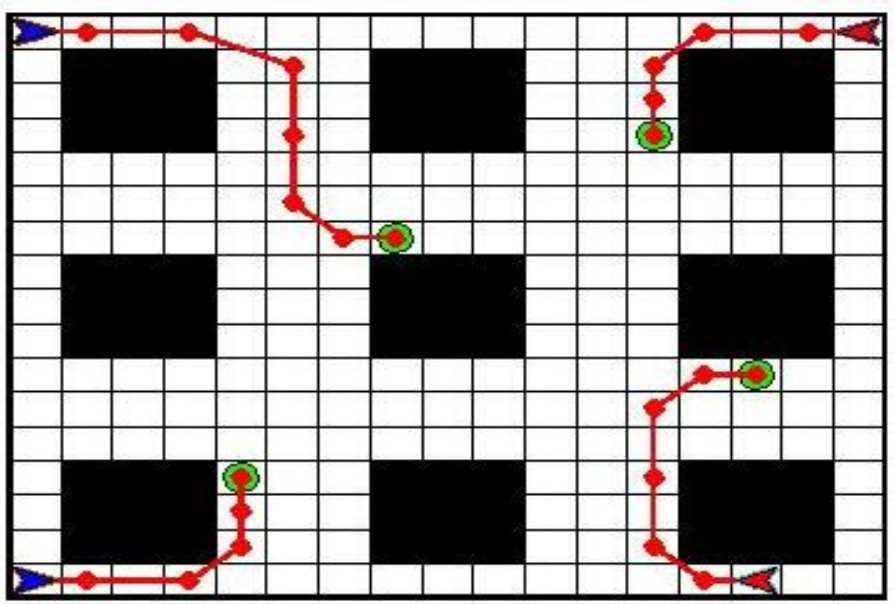


Figure: CSP with Quadcopter dynamics

The Problem

- Two variants of CSPP
  - **FindNCS:** Find an optimal number of charging stations (*ncs*) given a battery threshold (*d*)
  - **FindD:** Find an optimal battery threshold (*d*) for a given number of charging stations (*ncs*)

Charging station placement

- FindNCS:
  - Minimize the number of charging stations *ncs* and find their locations $CS = \{l_{c_1}, \ldots, l_{c_{ncs}}\}$ in the workspace
  - Such that a robot can reach the charging station from any obstacle-free location $l_f \in F$ by executing at most d number of transitions.

free space → given

$$\begin{aligned} &\texttt{minimize} && ncs \\ &\texttt{subject to} && \forall l_f \in F.\ \exists \xi = (\sigma_0 \ldots \sigma_d) \texttt{ with} \\ &&& \sigma_0.x = l_f \ \wedge \\ &&& (\sigma_d.x = l_{c_1} \vee \ldots \vee \sigma_d.x = l_{c_{ncs}}) \ \wedge \\ &&& \sigma_d.v = v_0 \end{aligned}$$

trajectory

initial

final

$l_{c_{ncs}}$ → ncs is unknown

state = $\langle$ loc x , vel v $\rangle$

Charging Station Placement

Charging Station Placement

- NP-hardness

Charging Station Placement

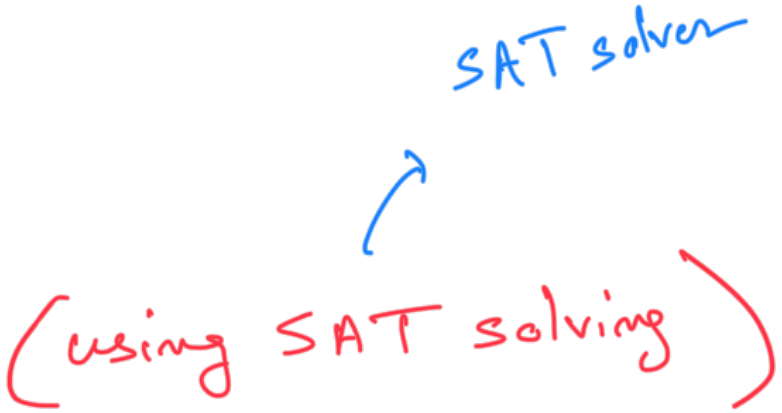
- NP-hardness
  - FindNCS: Polynomial time reduction from the NP-hard *Dominating Set* problem

Charging Station Placement

- NP-hardness
  - FindNCS: Polynomial time reduction from the NP-hard *Dominating Set* problem
  - FindD: Polynomial time reduction from the NP-hard *Vertex Cover* problem

Charging Station Placement

- NP-hardness
  - FindNCS: Polynomial time reduction from the NP-hard *Dominating Set* problem
  - FindD: Polynomial time reduction from the NP-hard *Vertex Cover* problem
- **Tackling NP-hardness**
  - Counter-example guided approach



1. **Charging station placement**
   - **Counter-example guided approach**

SMT-based Approach

SMT-based Approach

- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem

SMT-based Approach

- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem
- Input parameters: F, O, Γ, d (or ncs) .

SMT-based Approach

- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem
- Input parameters: F, O, Γ, d (or *ncs*) .
  - *Motion primitive* (γ)*:* A controllable action performed by the robot in each discrete time step.

SMT-based Approach

- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem
- Input parameters: F, O, Γ, d (or *ncs*) .
  - *Motion primitive* (γ)*:* A controllable action performed by the robot in each discrete time step.
  - Execution of $\gamma \in \Gamma$ is subject to *pre-condition* $pre(\gamma)$, *post-condition* $post(\sigma, \gamma)$ and other constraints (obstacle and collision avoidance)

SMT-based Approach

- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem
- Input parameters: F, O, Γ, d (or ncs) .
  - *Motion primitive* (γ): A controllable action performed by the robot in each discrete time step.
  - Execution of $\gamma \in \Gamma$ is subject to *pre-condition* $pre(\gamma)$, *post-condition* $post(\sigma, \gamma)$ and other constraints (obstacle and collision avoidance)
- Decision parameters:

SMT-based Approach

- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem
- Input parameters: F, O, Γ, d (or *ncs*) .
  - *Motion primitive* (γ)*:* A controllable action performed by the robot in each discrete time step.
  - Execution of $\gamma \in \Gamma$ is subject to *pre-condition* $pre(\gamma)$, *post-condition* $post(\sigma, \gamma)$ and other constraints (obstacle and collision avoidance)
- Decision parameters:
  - Number of charging stations (*ncs*) and Battery threshold (d) for FindNCS and FindD respectively.

SMT-based Approach

- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem
- Input parameters: F, O, Γ, d (or ncs) .
  - *Motion primitive (γ):* A controllable action performed by the robot in each discrete time step.
  - Execution of $\gamma \in \Gamma$ is subject to *pre-condition* $pre(\gamma)$, *post-condition* $post(\sigma, \gamma)$ and other constraints (obstacle and collision avoidance)
- Decision parameters:
  - Number of charging stations (ncs) and Battery threshold (d) for FindNCS and FindD respectively.
  - Charging station locations ($CS \subset F$ and $CS = \{l_{cs}^1, l_{cs}^2, \dots\}$)

SMT-based Approach

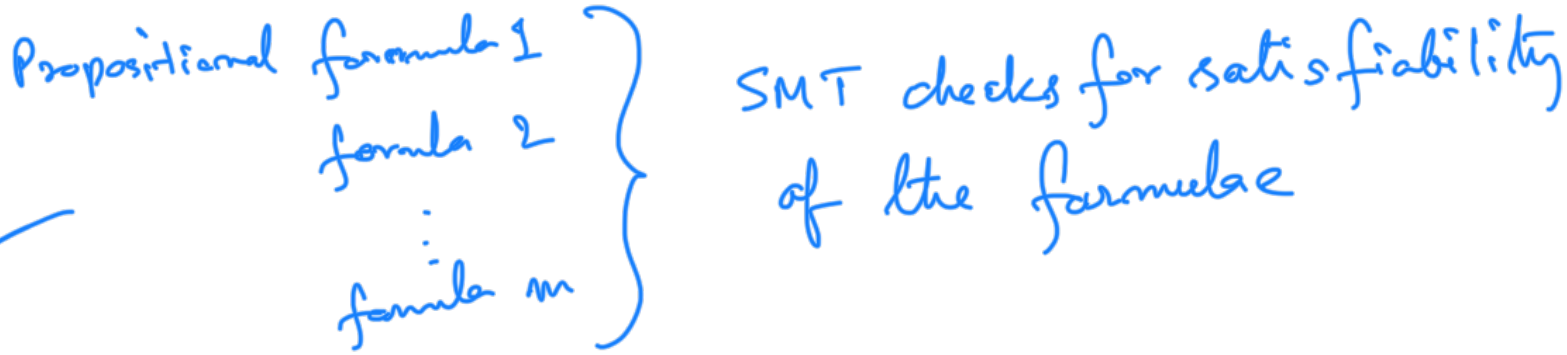
- CSP is reduced to a *Satisfiability Modulo Theories (SMT)* solving problem
- Input parameters: F, O, Γ, d (or ncs) .
  - *Motion primitive (γ):* A controllable action performed by the robot in each discrete time step.
  - Execution of $\gamma \in \Gamma$ is subject to *pre-condition* $pre(\gamma)$, *post-condition* $post(\sigma, \gamma)$ and other constraints (obstacle and collision avoidance)
- Decision parameters:
  - Number of charging stations (ncs) and Battery threshold (d) for FindNCS and FindD respectively.
  - Charging station locations ($CS \subset F$ and $CS = \{l_{cs}^1, l_{cs}^2, \dots\}$)
  - Motion plan from each $l_i \in F$. For l_i, motion plan $\rho_i = (\gamma_1 \dots \gamma_d)$

SMT-based Approach – Basic Algorithm

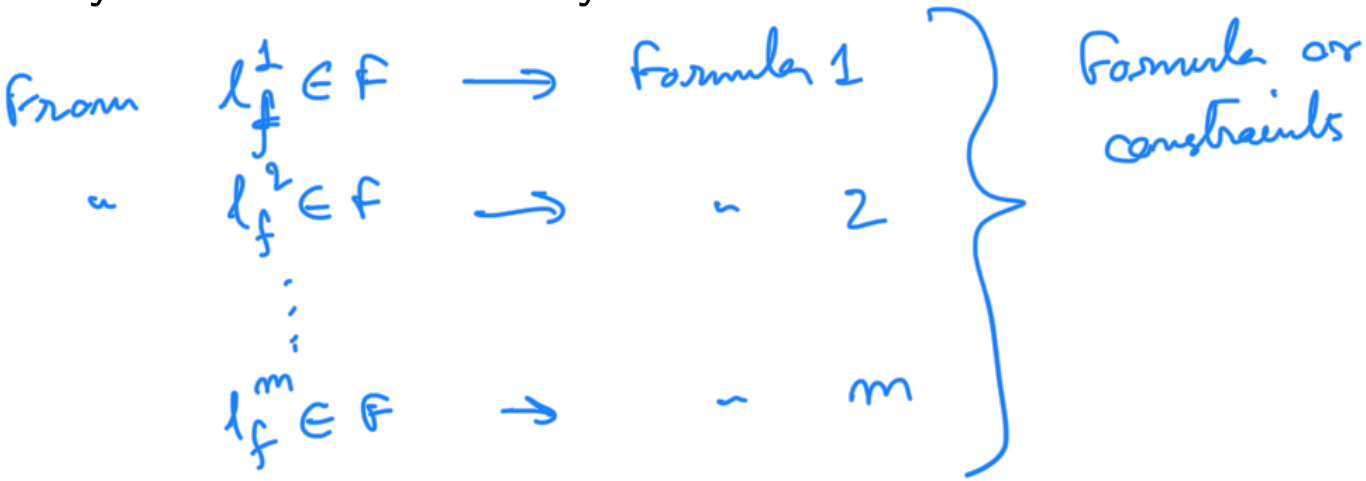
- Simply encodes the constraints
- Provides optimal solution when terminates
- **Scalability issue** with increasing state space (due to larger worksoaces and complex robot dynamics)



SMT-based Approach – UNSAT Core based Algorithm



- **Unsatisfiable core :** In an unsatisfiable propositional formula, a subset of clauses whose conjunction is still unsatisfiable.
- **Unsat-core based algorithm** for FindNCS
  - Significantly increases scalability.



Example of UNSAT-Core based CSP with Turtlebot, $d = 5$ and $\delta = 0$